

Step 4 — A Verifier Agent That Catches the Worker's Bad Answers — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper LangChain), **Advanced** (what makes the Verifier a genuine independent check instead of a rubber stamp). Read Basic first even if you already know the pattern — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

The Verifier is just another LCEL chain — structurally, it's Step 1 all over again: prompt in, model call, parsed answer out. It has no tools, and it isn't part of the Worker's loop at all — it runs once, after the Worker is completely finished, and looks at exactly two things: the original question, and the Worker's final answer.

Its job is narrow on purpose: does this answer actually address the question, and does it look consistent with what a Worker with tools should have been able to find? It never sees the Worker's step-by-step log, and it has its own separate system prompt — it isn't "the Worker, but told to double-check itself," it's a genuinely different role.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The function to build:

```
def verify(question: str, worker_answer: str) -> VerifierVerdict:
```

`VerifierVerdict` should be a Pydantic model with at least `approved: bool` and `reason: str` — not a bare string, because `main.py` needs to make a real decision (retry, or show the flag) based on the outcome, and parsing free text for "did it approve or not" is exactly the kind of fragile string-matching Doc01's error-handling philosophy warns against.

Structure the Verifier's own prompt around 2 explicit questions, matching the README: "does this answer actually address the question asked?" and "does this look consistent with something a tool-using agent could really have found, or does it read like an unsupported guess?" Use LangChain's structured output (`model.with_structured_output(VerifierVerdict)`) so the model's response is parsed directly into your Pydantic shape instead of you regex-ing a raw string.

Wire the main flow in `main.py`: run the Worker to completion, pass its `AgentResult.answer` and the original task into `verify()`, and branch on `verdict.approved`.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

The README is explicit about the trap here: "a Verifier that always approves isn't really tested — it's just for show." That's not just a testing footnote, it points at the real design risk. A Verifier whose system prompt is even slightly too agreeable ("review this answer and note any issues") will tend to find the Worker's answer plausible almost every time, because most answers *are* plausible-sounding even when they're wrong — plausibility is not the same thing as being actually checked against the question and the evidence. Write the Verifier's prompt to make rejection the *comfortable* default when it's genuinely unsure, not approval — something closer to "if you cannot confirm this answer is well-supported, say so; don't give the benefit of the doubt."

The second real gap: what happens when the Verifier rejects? The README gives you a real choice — retry the Worker with the Verifier's feedback attached, or surface the flag to the user instead of the answer. A silent retry loop with no limit of its own reintroduces exactly the problem Step 3 just solved for the Worker — an unbounded loop, just one level up. Cap the number of Worker → Verifier round-trips (2 is plenty for this exercise), and if it's still rejected after that many attempts, show the flag rather than looping forever hoping the next attempt is better.

The extra pieces: - A system prompt that explicitly instructs "reject if unsure," not "approve unless clearly wrong." - `verify(question, worker_answer, worker_steps_summary=None)` — an optional, *lightly* summarized view of what the Worker actually did (like "called: calculator, lookup_weather" — not the full raw log), enough for the Verifier to sanity-check "did this answer plausibly come from real tool use," without handing it the full transcript that would let it grade effort instead of the answer. - A `max_verify_attempts` cap in `main.py`'s loop, with a clear message shown to the user if it's exhausted.

Design your rejection-biased system prompt, and your retry cap, before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate get a structurally correct, independent second agent running. Advanced is about the one failure mode that makes an entire Verifier pointless without ever looking broken: an agreeable Verifier that passes everything, and a retry path with no exit that can spin forever the moment it doesn't.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
class VerifierVerdict: approved (bool), reason (str)

function verify(question, worker_answer):
    build a prompt: "question was X, answer given was Y – does this
        actually answer the question, and does it look well-supported?"
    ask the model, get back a structured VerifierVerdict
    return it

in main.py:
    result = run_agent(task)
    verdict = verify(task, result.answer)
    if verdict.approved:
        show result.answer
    else:
        show the flag: verdict.reason
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
agents/verifier_agent.py:
class VerifierVerdict(BaseModel):
    approved: bool
    reason: str

VERIFIER_SYSTEM_PROMPT = """You are an independent verifier. You did not
produce this answer. Given a question and a proposed answer, judge two
things: (1) does the answer actually address the question, (2) does it
look consistent with real tool use rather than an unsupported guess."""

def verify(question: str, worker_answer: str) -> VerifierVerdict:
    model = ChatOpenAI(model="gpt-4o-mini").with_structured_output(VerifierVerdict)
    prompt = ChatPromptTemplate.from_messages([
        ("system", VERIFIER_SYSTEM_PROMPT),
        ("human", "Question: {question}\n\nProposed answer: {answer}"),
    ])
    chain = prompt | model
    return chain.invoke({"question": question, "answer": worker_answer})

main.py:
from agents.worker_agent import run_worker
from agents.verifier_agent import verify

result = run_worker(task, max_iterations=6)
verdict = verify(task, result.answer)
if verdict.approved:
    print(result.answer)
else:
    print(f"FLAGGED: {verdict.reason}")
```

Write the full typed version yourself — including logging both agents' output separately, as the README asks — before moving to the Advanced version.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

VERIFIER_SYSTEM_PROMPT adds: "If you cannot confirm the answer is well-supported, reject it. Do not give the benefit of the doubt."

```
main.py:
    attempt = 0
    max_verify_attempts = 2
    result = run_worker(task, max_iterations=6)
    while attempt < max_verify_attempts:
        verdict = verify(task, result.answer)
        if verdict.approved:
            show result.answer
            break
        attempt += 1
        if attempt >= max_verify_attempts:
            show the flag: verdict.reason
            break
        result = run_worker(task + f"\n\nA reviewer rejected your last answer: {v
                                max_iterations=6)
```

Here's almost the whole thing — fill in the missing piece yourself:

```
def run_worker_with_verification(task: str, max_verify_attempts: int = 2):
    result = run_worker(task, max_iterations=6)
    for attempt in range(max_verify_attempts):
        verdict = verify(task, result.answer)
        if verdict.approved:
            return result.answer, verdict
        if attempt == max_verify_attempts - 1:
            # your turn: this was the last allowed attempt – return the flag
            # instead of the answer, so the caller shows it, not a possibly-wrong
            ...
        # your turn: re-run the Worker, feeding the verdict's reason back into th
        ...
```

Fill in both marked pieces, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate build a real, independent second agent with a real handoff — that's most of what makes this "genuinely multi-agent." Advanced is the part that decides whether that second agent actually catches anything: a

rejection-biased prompt instead of an agreeable one, and a bounded retry instead of a loop that could, in principle, run forever the same way an unbounded Worker could in Step 2.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)